MÓDULO 5 — TOOLS, AGENTES, SKILLS Y MEMORIA · CLASE 9 DE 16

Tools, Function Calling y Agent Loop

C8 terminó con un LLM Router que podía decidir qué capacidad necesitaba. Hoy formalizamos esa decisión: el modelo propone una Tool y sus argumentos; la aplicación valida, autoriza, ejecuta y devuelve una observación. Al repetir ese ciclo aparece el Agent Loop. El RAG de las clases anteriores queda integrado como una capacidad más dentro de un sistema que puede decidir y actuar.

Function Calling · Tool Contract · JSON Schema · Pydantic · Tool Registry · Policy Gate · Agent Loop · Agentic RAG

Formalizar

Pasar del routing artesanal de C8 a llamadas estructuradas con nombre y argumentos.

Ejecutar con control

Separar propuesta del modelo de validación, permisos y ejecución real.

Cerrar el loop

Transformar resultados y errores en observaciones que alimentan la siguiente decisión.

NÚCLEO OBLIGATORIO

Tool Calling como contrato

Function Calling, schemas, Pydantic, validación y separación entre lo que propone el modelo y lo que ejecuta la aplicación.

PRÁCTICA DE CONSOLIDACIÓN

Runtime agéntico mínimo

Registry, policy, errores, límites y un loop capaz de usar retrieval, consulta externa y generación de reportes.

AMPLIACIÓN / LUEGO

Estado, HITL y memoria

C10 agrega Skills y memoria. C11 formaliza estado, checkpoints y aprobación humana para acciones sensibles.

Frontera de esta clase: hoy sí ejecutamos Tools y construimos el loop. No resolvemos todavía memoria de largo plazo ni workflows persistentes con aprobación/reanudación. Tampoco usamos MCP: la interoperabilidad estándar entre sistemas llega en C13.

GUÍA

Pregunta central y conexión curricular

PREGUNTA CENTRAL

¿Qué problema resolvemos?

En C8 el modelo podía clasificar una consulta como `search_course`, `calculator` o `external_lookup`. Hoy preguntamos: **¿cómo convertimos esa intención en una acción estructurada, validable y segura, y cómo usamos su resultado para decidir el siguiente paso?**

OBJETIVOS

Al finalizar deberías poder

- Explicar qué ocurre realmente durante un Function Call.
- Diseñar una Tool con nombre, descripción, schema, output y policy.
- Distinguir conformidad estructural de autorización de negocio.
- Separar Registry, disponibilidad y ejecución.
- Manejar errores, timeout, retries y límites del loop.
- Construir un agente mínimo con RAG como una Tool.

SECUENCIA

De C8 a C11

C8: decide qué capacidad necesita. **C9:** formaliza y ejecuta Tools dentro de un loop. **C10:** incorpora Skills, Context Engineering y memoria. **C11:** agrega estado persistente, checkpoints y HITL.

Definición operativa de agente para esta clase

Un agente es una aplicación en la que el modelo participa de la decisión sobre el próximo paso dentro de capacidades y límites definidos.

La autonomía está en la decisión; el control de ejecución sigue siendo responsabilidad del software.

PARTE 1

Del LLM Router al Tool Calling

La C8 ya tenía la semilla de un agente

El router de C8 hacía algo importante: el modelo ya no respondía siempre, sino que podía decidir qué capacidad correspondía. Pero el contrato era nuestro: devolvíamos un JSON propio y luego un bloque `if/elif` traducía esa ruta a código.



Del router artesanal de Clase 8 al Function Calling de Clase 9.. La trazabilidad comienza en la ingesta y continúa hasta las métricas.

CÓMO LEER ESTE VISUAL

QUÉ MUESTRA

El mismo problema resuelto de dos maneras: routing estructurado propio y Tool Calling formal.

CÓMO INTERPRETARLO

La diferencia no es que el modelo “gane acceso” a Python. Cambia el contrato de comunicación entre modelo y aplicación.

DECISIÓN DE INGENIERÍA

Usar Function Calling cuando necesitamos que la intención de actuar sea explícita, tipada y componible dentro del runtime.

C8 · ROUTER ARTESANAL

```
Opciones:
- search_course
- calculator
- external_lookup

Devolvé JSON:
{"route": "..."}
```

C9 · TOOLS DECLARADAS

```
tools = [
  search_course_schema,
  external_lookup_schema,
  generate_report_schema
]

El modelo puede proponer una tool_call.
```

La continuidad conceptual

C8 enseñó *selección semántica*. C9 no tira ese conocimiento: lo formaliza. El modelo sigue interpretando intención; ahora la aplicación le expone contratos de capacidades y recibe una propuesta estructurada de acción.

PARTE 2

Function Calling: propuesta estructurada, no ejecución automática

Vocabulario base de la clase

Antes de recorrer Function Calling conviene fijar cinco palabras que van a aparecer durante toda la clase. No son sinónimos: cada una nombra una responsabilidad distinta del sistema.

Capacidad lógica que la aplicación decide exponer al modelo para que pueda seleccionarla cuando la tarea la necesite.

Puede estar implementada con una función local, una base de datos, una API, otro modelo o una combinación de componentes. Tool no significa API.

Propuesta estructurada del modelo que indica qué Tool quiere usar y con qué argumentos.

Todavía no ocurrió ninguna acción: la aplicación debe validar y autorizar la propuesta antes de ejecutar.

Código real de la aplicación que implementa una Tool.

La Tool search_course puede tener como handler a la función Python que consulta el retriever construido en C5–C8.

Representación del resultado de ejecutar una Tool —dato, metadata o error— que vuelve al contexto para que el modelo pueda decidir el siguiente paso.

No es un "pensamiento" del modelo: es información producida por el runtime después de una acción.

Capa de software que controla el ciclo del agente: llama al LLM, recibe sus decisiones, valida y autoriza Tool Calls, ejecuta handlers, registra observations y aplica límites.

El LLM propone el próximo paso; el runtime conserva el control de ejecución y decide qué ocurre con esa propuesta.

QUIÉN LLAMA A QUIÉN

El LLM decide; el Runtime controla la ejecución

El modelo no queda "corriendo" ni ejecuta funciones por sí solo. En cada paso, el runtime hace una llamada al LLM, recibe una propuesta y decide qué hacer con ella.

01 · RUNTIME

Llama al LLM

mensajes + Tools permitidas

↓

02 · LLM

Propone

respuesta final o Tool Call

↓

03 · RUNTIME

Valida + autoriza

Registry · schema · Policy

↓

04 · HANDLER

Ejecuta

código real de la Tool

↓

05 · OBSERVATION

Resultado o error

vuelve al contexto

↺ **Con la Observation, el Runtime actualiza el contexto y vuelve a llamar al LLM. Cada nueva decisión es una nueva llamada al modelo.**

¡ **El patrón es independiente del proveedor.**
Gemini, OpenAI, Anthropic u otros proveedores pueden representar Tools y Tool Calls con APIs o estructuras diferentes. En el curso nos importa el patrón arquitectónico común: **declarar capacidades → recibir una propuesta estructurada → validar y autorizar → ejecutar → devolver la observación**.

La idea más importante de la clase

Function Calling permite que el modelo produzca una **intención de llamada estructurada**: qué Tool quiere usar y con qué argumentos. Eso no significa que el modelo ejecute la función ni que tenga acceso arbitrario al runtime.



Ciclo completo de Function Calling con separación entre modelo y aplicación.. Abrir visual ampliado.. La trazabilidad comienza en la ingesta y continúa hasta las métricas.

CÓMO LEER ESTE VISUAL

QUÉ MUESTRA

El recorrido completo desde la decisión del modelo hasta la observación que vuelve al contexto.

CÓMO INTERPRETARLO

Hay una frontera deliberada entre propuesta y ejecución. En esa frontera viven validación, permisos y políticas.

DECISIÓN DE INGENIERÍA

Nunca despachar directamente argumentos generados por el modelo a una operación real sin validarlos y autorizarlos.

Ejemplo conceptual

LECTURA GUIADA

JSON

```
{
  "name": "search_course",
  "arguments": {
    "query": "¿qué diferencia hay entre dense y hybrid retrieval?",
    "top_k": 5
```

```
    }
}
```

Cómo leerlo

El modelo no devolvió la respuesta final. Propuso usar una capacidad y completó sus argumentos. La aplicación todavía tiene que comprobar que `search_course` existe, que `top_k` es válido, que esa Tool está disponible y que puede ejecutarse en este contexto.

Function Calling no reemplaza Structured Outputs

Son patrones relacionados pero distintos. Un Structured Output controla la forma de una **respuesta del modelo**. Un Tool Call expresa una **intención de acción** que la aplicación puede aceptar o rechazar. En ambos casos, el schema reduce ambigüedad.

MODELO	SCHEMA
propone	**estructura**
APLICACIÓN	**HANDLER**
autoriza	**ejecuta**

PARTE 3

Tool Contract: de una función Python a una capacidad profesional

La Tool de C8 ahora necesita un contrato completo

En C8 vimos implementación, interfaz y contrato semántico. En C9 agregamos la parte operacional: schema de argumentos, output estructurado, errores, riesgo, timeout y política de ejecución.

Anatomía profesional de una Tool: contrato visible, handler y controles. La trazabilidad comienza en la ingesta y continúa hasta las métricas.

CÓMO LEER ESTE VISUAL

QUÉ MUESTRA

La cara que interpreta el modelo y la cara que controla el runtime.

CÓMO INTERPRETARLO

Una descripción perfecta no reemplaza un schema; un schema válido no reemplaza permisos; y un handler correcto no define por sí mismo cuándo debe ejecutarse.

DECISIÓN DE INGENIERÍA

Diseñar cada Tool como un contrato completo, no como una función decorada.

Qué significa execution policy

Una
policy
es un conjunto de reglas confiables de la aplicación que determina si una capacidad está disponible y bajo qué condiciones puede ejecutarse. No la decide el LLM ni puede ser modificada por texto recuperado. Puede considerar identidad, rol, contexto, riesgo, presupuesto, necesidad de aprobación y restricciones de negocio.

Pydantic como frontera de validación

LECTURA GUIADA

PYTHON

```python
from pydantic import BaseModel, Field

class SearchCourseArgs(BaseModel):
    query: str = Field(min_length=3, max_length=500)
    top_k: int = Field(default=5, ge=1, le=10)

class SearchCourseResult(BaseModel):
    chunks: list[dict]
    strategy: str
    took_ms: int
```

! **Schema válido ≠ acción autorizada**
Pydantic puede garantizar que `top_k=5` tiene el tipo y rango correctos. No puede decidir si el usuario tiene permiso para consultar ese corpus, si la Tool está habilitada en esta sesión o si una acción externa requiere aprobación.

La descripción sigue siendo parte del comportamiento

Descripción pobre

Ambigua la selección

`"Busca información."`

No define dominio, fuente, límites ni cuándo preferir otra capacidad.

Descripción útil

Reduce ambigüedad

`"Busca evidencia en el corpus del curso AI Agent Developer. Usar cuando la respuesta depende del material de clases. No usar para datos actuales ni cálculos determinísticos."`

Output contract: también hay que diseñar el regreso

Una Tool útil no debería devolver una mezcla impredecible de strings, excepciones y objetos. Conviene acordar una forma estable para éxito y error, de modo que el runtime pueda transformarla en una observation interpretable.

LECTURA GUIADA — JSON

```json
{
  "ok": true,
  "data": { "chunks": [ ... ], "strategy": "hybrid_rerank" },
  "error": null
}
```

PARTE 4

Tool Registry + Policy Gate: catálogo no es permiso

Separar tres preguntas distintas

1. **¿Qué Tools existen?** Registry.
2. **¿Qué Tools puede ver este modelo en este contexto?** Availability / Policy.
3. **¿Esta llamada concreta puede ejecutarse?** Authorization + validation.

Concepto	Pregunta que responde	Ejemplo
Registry	¿La capacidad existe en la aplicación?	`generate_report` está implementada y registrada.
Availability	¿Conviene exponerla al modelo en este contexto?	A un alumno puede no mostrársele `generate_report`; a un docente sí.

Validation	¿Los argumentos tienen forma, tipos y restricciones válidas?	`top_k=500` se rechaza si el máximo es 10.
Authorization	¿Este actor puede realizar esta operación concreta sobre este recurso?	Un docente puede ver la Tool pero no generar un reporte de un curso al que no tiene acceso.

Disponibilidad no es autorización.

Que una Tool sea visible para el modelo reduce el espacio de decisión. La autorización se vuelve a comprobar sobre la llamada concreta, porque los argumentos pueden cambiar qué recurso, usuario o acción está involucrado.

Tool Registry y Policy Gate separan capacidades registradas de capacidades disponibles.. La trazabilidad comienza en la ingesta y continúa hasta las métricas.

CÓMO LEER ESTE VISUAL

QUÉ MUESTRA

Un catálogo de capacidades atravesado por una capa de política antes de exponer Tools al modelo.

CÓMO INTERPRETARLO

El conjunto de Tools puede variar según usuario, rol, contexto, etapa del workflow o nivel de riesgo.

DECISIÓN DE INGENIERÍA

Calcular `allowed_tools(context)` antes de invocar el modelo y volver a autorizar la llamada concreta antes de ejecutarla.

Registry mínimo

LECTURA GUIADA PYTHON

```python
TOOLS = {
    "search_course": ToolSpec(
        args_model=SearchCourseArgs,
        handler=search_course,
        risk="read",
        timeout_s=5,
    ),
    "external_lookup": ToolSpec( ... ),
    "generate_report": ToolSpec( ... , risk="side_effect"),
}
```

Acceso mínimo necesario

Si un agente está respondiendo una duda académica, no necesita recibir una Tool capaz de enviar correos o modificar registros. Menos Tools visibles significa menor superficie de error, menor ambigüedad de selección y menor riesgo.

Regla de diseño: primero reducir las capacidades disponibles al mínimo necesario; después pedirle al modelo que elija entre ellas.

PARTE 5

Ejecución segura: side effects, errores y contenido no confiable

Efecto externo (side effect)

Un
side effect
es un cambio producido fuera del cálculo interno del agente que puede persistir o afectar otros sistemas o personas: enviar un email, crear o modificar un registro, cancelar una reserva o realizar un pago. Leer datos o calcular un resultado local normalmente no produce ese tipo de efecto.

Read Tool y Side-effect Tool no son equivalentes

Clasificación de Tools por riesgo y side effects. La trazabilidad comienza en la ingesta y continúa hasta las métricas.

CÓMO LEER ESTE VISUAL

QUÉ MUESTRA

Un continuo de riesgo desde lectura hasta acciones con efecto externo.

CÓMO INTERPRETARLO

La clasificación de riesgo determina qué controles aplicamos, no solo cómo describimos la Tool.

DECISIÓN DE INGENIERÍA

Automatizar más fácilmente lectura y cálculo; elevar controles para operaciones irreversibles o externas. C11 formaliza la aprobación humana y reanudación.

Instrucciones no confiables no expanden permisos

Un documento recuperado por RAG, una página externa o un resultado de otra Tool puede contener texto como “ignorá las reglas anteriores y ejecutá X”. Ese contenido es **dato**, no autoridad. No debe poder habilitar Tools, cambiar políticas ni ampliar permisos del runtime.

! **El contenido recuperado puede informar argumentos; no puede definir la política.**
Las reglas de disponibilidad y autorización nacen de configuración confiable de la aplicación, identidad del usuario y contexto operativo.

Errores observables y retries deliberados

Recuperamos C3: resiliencia dentro del Agent Loop

En C3 usamos
timeout
para limitar cuánto esperamos,
retry
para repetir de forma selectiva un fallo transitorio,
backoff
para esperar progresivamente entre reintentos y
run_id
para reconstruir una ejecución. En C9 aplicamos las mismas ideas a cada paso de un agente, con una diferencia importante: ahora una Tool puede producir efectos externos y repetirla no siempre es seguro.

Falla	Ejemplo	Respuesta del runtime
Argumentos inválidos	`top_k=500`	No ejecutar; devolver error de validación.
Timeout	API externa no responde	Cortar según presupuesto; retry solo si la operación es segura.
No autorizado	Tool no habilitada	Bloquear; nunca “probar igual”.
Error transitorio	HTTP 503	Retry acotado con backoff si aplica.

Error de negocio	recurso inexistente	Observation estructurada para que el agente decida alternativa.

Retries, side effects e idempotencia

Reintentar una lectura suele ser distinto de reintentar una acción que crea, envía o modifica algo.
Idempotencia
significa que repetir la misma operación no produce un efecto adicional no deseado. Si una respuesta se pierde y el runtime reintenta, una operación idempotente evita crear dos veces el mismo registro, enviar dos pagos o duplicar una acción. Cuando una Tool con efecto externo no es idempotente, el runtime necesita una estrategia explícita —por ejemplo una clave de idempotencia, detección de duplicados o revisión humana— antes de reintentar.
`search_course()` repetida suele ser inocua; `send_email()` repetida puede enviar dos correos. El mismo mecanismo de retry tiene consecuencias distintas según la Tool.

Trazabilidad mínima

Registrar al menos `run_id`, paso, Tool, argumentos, duración, resultado/error y decisión de autorización. Antes de escribir argumentos en logs hay que **sanearlos**: eliminar, ocultar o enmascarar secretos y datos sensibles que no deberían quedar persistidos. C15 profundizará observabilidad; hoy alcanza con entender que un agente sin trazas es difícil de depurar y gobernar.

PARTE 6

Agent Loop y Agentic RAG

El loop aparece cuando una Tool no necesariamente termina la tarea

El resultado de una Tool vuelve al modelo como **observation**: información producida por el runtime después de la acción, como datos obtenidos, metadata relevante o un error estructurado. Con esa nueva evidencia, el modelo puede responder, pedir otra Tool o corregir una decisión anterior. Ese ciclo es la diferencia central respecto del router de una sola pasada.

Agent Loop con decisiones, acciones, observaciones y límites.. La trazabilidad comienza en la ingesta y continúa hasta las métricas.

CÓMO LEER ESTE VISUAL

QUÉ MUESTRA

El ciclo mínimo Decide → Act → Observe y sus condiciones de stop.

CÓMO INTERPRETARLO

La autonomía está acotada por el runtime: cada vuelta consume tiempo y costo, y cada acción vuelve a pasar por validación y policy.

DECISIÓN DE INGENIERÍA

Definir presupuestos y finalización antes de ejecutar el loop, no después de que se descontrole.

Qué significa detener el loop

Stop condition	Qué significa
`final_answer`	El modelo considera que ya tiene información suficiente para responder al usuario.
`max_steps`	Se alcanzó el máximo de iteraciones permitido aunque la tarea no haya terminado.
`timeout`	La ejecución superó el tiempo total aceptable.
`budget`	Se agotó un presupuesto definido para la ejecución: puede expresarse en pasos, llamadas, tokens, tiempo o costo.
`unrecoverable_error`	Ocurrió un error del que el agente no puede recuperarse razonablemente mediante otra decisión o un retry.

Regla de diseño: un agente no es un `while True`. La autonomía siempre vive dentro de límites definidos por el runtime.

El Agent Loop es un loop del Runtime, no del LLM

El LLM se ejecuta mediante llamadas discretas. En cada iteración, el runtime construye el contexto, llama al modelo y recibe una nueva decisión. Si hay un Tool Call, el runtime lo valida, aplica Policy, ejecuta el handler, agrega la Observation y recién entonces realiza otra llamada al LLM. En el pseudocódigo siguiente, cada `model.decide(...)` representa una nueva llamada al modelo.

Pseudocódigo mínimo del runtime

LECTURA GUIADA

PYTHON

```python
for step in range(MAX_STEPS):
    allowed = policy.allowed_tools(context)
    decision = model.decide(messages, tools=allowed)

    if decision.is_final_answer:
        return decision.answer

    spec = registry[decision.tool_name]
    args = spec.args_model.model_validate(decision.arguments)
    policy.authorize(context, spec, args)

    observation = execute_with_limits(spec, args)
    messages.append(observation)

raise AgentLimitError("max_steps reached")
```

Qué mirar, no qué memorizar

- El modelo ve solo las Tools permitidas.
- La validación ocurre antes del handler.
- La policy autoriza la llamada concreta.
- El resultado se agrega como observation.
- El loop tiene un límite explícito.

Agentic RAG: todo C5–C8 pasa a ser una capacidad

Agentic RAG no es otro algoritmo de retrieval

Es una arquitectura en la que
recuperar información es una de las acciones que el agente puede elegir
dentro de un proceso de múltiples pasos. Dense, Hybrid Search, RRF y reranking describen cómo recuperamos; Agentic RAG describe
quién decide si recuperar y en qué momento del flujo
.

Agentic RAG con retrieval, consulta externa y generación de reporte como Tools.. La trazabilidad comienza en la ingesta y continúa hasta las métricas.

CÓMO LEER ESTE VISUAL

QUÉ MUESTRA

Un agente que puede usar retrieval, una consulta externa o una Tool de generación.

CÓMO INTERPRETARLO

El RAG no desaparece: gana una frontera clara y puede ser elegido solo cuando la tarea requiere conocimiento del corpus.

DECISIÓN DE INGENIERÍA

Mantener `search_course()` estable aunque cambie internamente dense, hybrid o reranking. El agente depende del contrato, no de la implementación.

Del router de una pasada al agente

C8 · Router

Una decisión

1. Clasificar intención.
2. Elegir una capacidad.
3. Ejecutarla.
4. Generar respuesta.

C9 · Agent Loop

Decisiones iterativas

1. Decidir próximo paso.
2. Proponer Tool Call.
3. Validar y ejecutar.
4. Observar resultado.
5. Decidir nuevamente o finalizar.

NOTEBOOK

Notebook guiada — construir el runtime antes del framework

PRÁCTICA TÉCNICA

La notebook debe hacer visible cada frontera del sistema

En lugar de empezar con un helper que oculta el loop, construimos primero un runtime pequeño donde podamos inspeccionar tool calls, validación, policy, handlers, observations y condiciones de stop.

01 · CONTRATO

ToolSpec

`name · description · schema`

↓

02 · RUNTIME

Registry + Policy

`allowed_tools()`

↓

03 · EJECUCIÓN

validate + call

`timeout · errors`

↓

04 · LOOP

observe + decide

`MAX_STEPS`

Tres Tools obligatorias del laboratorio

RETRIEVAL

`search_course()`

Reutiliza la Tool optimizada en C8 y devuelve evidencia con metadata.

CONSULTA EXTERNA

`external_lookup()`

Consulta una fuente externa o mock controlado y devuelve un resultado vigente/estructurado.

PRODUCCIÓN

`generate_report()`

Construye un reporte a partir de evidencia previa. En clase puede trabajar en modo local/dry-run.

Mock controlado

Probar sin depender del sistema real

Un **mock** es un sustituto controlado de un servicio o dependencia real que devuelve respuestas conocidas. Permite probar selección, validación, errores y observations aunque la API externa no esté disponible.

Dry-run

Recorrer la lógica sin producir el efecto real

Un **dry-run** ejecuta o simula el flujo suficiente para mostrar qué haría el sistema, pero evita persistir o disparar la acción externa. Es útil para probar Tools de mayor riesgo antes de habilitarlas en modo real.

Checkpoints

DEFINITION OF DONE

- [ ] Puedo inspeccionar el schema que recibe el modelo.
- [ ] Un argumento inválido se rechaza antes de ejecutar el handler.
- [ ] Una Tool no permitida se bloquea aunque el modelo la proponga.
- [ ] Errores y timeouts vuelven como observaciones controladas.
- [ ] El agente puede usar search_course y luego otra Tool antes de responder.
- [ ] El loop termina por respuesta final o por un límite explícito.

Después de entender el runtime

Recién entonces tiene sentido comparar cómo un framework encapsula el mismo ciclo. El objetivo pedagógico es poder reconocer qué abstracción está ocultando.

ACTIVIDAD

Tool Contract Review — diseñar para que otro agente no se equivoque

TRABAJO GRUPAL

Diseñá una Tool que otro equipo pueda integrar sin ver su implementación

Cada grupo recibe una capacidad de negocio. Debe producir el contrato visible al modelo y las reglas de ejecución del runtime. Otro grupo actuará como revisor y buscará ambigüedades, argumentos peligrosos y permisos implícitos.

1 · MODEL-FACING

Contrato semántico

- name
- description
- cuándo usar / no usar
- input schema

2 · RUNTIME-FACING

Contrato operacional

- handler
- output/error
- timeout/retry
- risk + policy

3 · RED TEAM

Casos adversos

- argumento inválido
- Tool equivocada
- contenido no confiable
- reintento peligroso

Canvas de entrega

Elemento	Decisión del grupo	Riesgo si está mal definido

Nombre	
Descripción	
Input schema	
Output contract	
Risk / side effect	
Policy	
Error / timeout	
Stop / fallback	

Regla

El equipo revisor solo puede ver el contrato de la Tool. Si necesita leer el código para saber cuándo usarla o qué riesgo tiene, el contrato está incompleto.

RECURSOS

Para profundizar sin adelantar otras clases

CONCEPTO

Function Calling

Contrato estructurado para que el modelo proponga una capacidad y sus argumentos; ejecución siempre del lado de la aplicación.

Tools Schemas

VALIDACIÓN

Pydantic + reglas de negocio

Validar forma y tipos es necesario, pero la autorización y las restricciones de negocio pertenecen a otra capa.

Validation Policy

ARQUITECTURA

Agent Loop

Decide → Act → Observe con límites explícitos de pasos, tiempo, costo y condiciones de finalización.

Runtime Control

PRÓXIMAS CLASES

Skills, memoria, estado y HITL

C9 construye la acción. C10 agrega capacidades reutilizables y continuidad; C11 hace el workflow persistente y supervisable.

C10 C11

CIERRE

Qué tiene que quedar de la Clase 9

01

El modelo propone

Un Tool Call es una intención estructurada; no acceso directo a código.

02

La aplicación controla

Valida schema, negocio, permisos, riesgo y disponibilidad antes de ejecutar.

03

Una Tool es un contrato

Descripción, argumentos, outputs, errores y policy son parte de la capacidad.

04

Los errores también son observations

El runtime transforma fallas controlables en información para la próxima decisión.

05

El loop necesita límites

Pasos, timeout, presupuesto y stop conditions evitan agentes infinitos.

06

RAG es una Tool

Todo C5–C8 queda disponible como search_course() dentro de un sistema agéntico.

Próxima clase

Clase 10 — Skills, contexto y memoria: cómo darle capacidades reutilizables y continuidad a un agente que ya sabe decidir y actuar.